

QREntropy

A Framework for Quantum Random Number Generation
and Randomness Certification

Anika Tabassum

MSc Student, Quantum Information Science
University of Copenhagen

June 2026

Research Question: How do quantum noise channels affect the statistical quality and entropy of quantum-generated random numbers, and can randomness extraction recover high-quality randomness from noisy quantum measurements?

0 Contents

1	Project Overview	3
2	Background and Motivation	3
2.1	Why Quantum Randomness?	3
2.2	The Problem of Noise	3
2.3	Project Repository Structure	4
3	Module 1 — Multiple QRNG Protocols	4
3.1	A. Hadamard QRNG	4
3.2	B. Bell-State QRNG	4
3.2.1	CHSH Inequality	5
3.3	C. Angle-Based QRNG	5
4	Module 2 — Noise Modeling	5
4.1	Quantum Noise Channels	6
4.1.1	Bit-Flip Channel	6
4.1.2	Phase-Flip Channel	6
4.1.3	Depolarizing Channel	6
4.1.4	Amplitude Damping	6
4.1.5	Measurement Error	6
4.2	Noise vs. Entropy: Research Question	6
5	Module 3 — Entropy Estimation	7
5.1	Shannon Entropy	7
5.2	Min-Entropy	7
5.3	Collision Entropy (Rényi Entropy of Order 2)	7
5.4	Rényi Entropy Family	7
6	Module 4 — Randomness Extraction	8
6.1	Von Neumann Extractor	8
6.2	SHA-256 Hash Extractor	8
6.3	Comparison: Raw vs. Extracted Entropy	8
7	Module 5 — Statistical Test Suite	8
7.1	Frequency / Monobit Test	8
7.2	Chi-Square Goodness-of-Fit Test	9
7.3	Runs Test	9
7.4	Autocorrelation Test	9
7.5	Serial Test (Digram Test)	9
7.6	Poker Test	9
7.7	NIST SP 800-22 Alignment	9
8	Module 6 — IBM Quantum Integration	9
8.1	Simulator Backend	10
8.2	IBM Quantum Hardware Backend	10
8.3	Simulator vs. Hardware Comparison	10
9	Module 7 — Interactive Dashboard	10
10	Module 8 — Visualization	11

11 Module 9 — Benchmarking	11
12 Module 10 — Research Component	11
12.1 Primary Research Question	11
12.2 Experimental Design	12
12.3 Expected Outcomes	12
12.4 Connection to Quantum Error Correction	12
13 Summary and Future Directions	12

1 Project Overview

QREntropy is a modular Python framework for studying quantum random number generation (QRNG) and randomness certification. It implements multiple QRNG protocols on both simulated and real IBM Quantum hardware, models realistic quantum noise channels, estimates information-theoretic entropy measures, applies randomness extraction, and evaluates output quality through a statistical test suite.

The framework is structured around ten interconnected modules:

1. **Multiple QRNG Protocols** — Hadamard, Bell-state, and angle-based generation
2. **Noise Modeling** — Bit-flip, phase-flip, depolarizing, amplitude damping, measurement error
3. **Entropy Estimation** — Shannon, min-entropy, collision entropy
4. **Randomness Extraction** — Von Neumann extractor, SHA-256 hashing
5. **Statistical Test Suite** — Chi-square, runs, autocorrelation, and more
6. **IBM Quantum Integration** — Simulator and real hardware backends
7. **Interactive Dashboard** — Streamlit-based user interface
8. **Visualization** — Histograms, entropy plots, heatmaps, circuit diagrams
9. **Benchmarking** — Comparison against classical pseudorandom generators
10. **Research Component** — Systematic noise vs. entropy study

2 Background and Motivation

2.1 Why Quantum Randomness?

Randomness is a foundational resource in cryptography, simulation, and scientific computing. Classical pseudorandom number generators (PRNGs) produce deterministic sequences that are computationally indistinguishable from random, but are fundamentally predictable given knowledge of the seed. Physical (hardware) random number generators exploit thermal noise or shot noise, but rely on assumptions about the entropy source.

Quantum random number generators offer a qualitatively different guarantee: the outcomes of quantum measurements are *intrinsically* random, a consequence of the Born rule. No hidden-variable model (within the constraints of Bell’s theorem) can predict the measurement outcome of a qubit in superposition.

Definition: Quantum Randomness

A measurement on a qubit prepared in the state $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ yields outcome 0 or 1, each with probability $\frac{1}{2}$, with no physical process capable of predicting which outcome occurs for any individual measurement.

2.2 The Problem of Noise

Real quantum devices are not ideal. Every gate operation and measurement introduces errors through interactions with the environment. These noise channels systematically bias the output distribution, reducing the effective entropy of the generated bits. A key research challenge is therefore:

- Characterising how different noise channels degrade randomness quality
- Applying randomness extraction (post-processing) to recover near-ideal randomness from noisy raw bits
- Comparing results between noise-free simulation and real quantum hardware

2.3 Project Repository Structure

```
QREntropy/
  src/
    generators/      # QRNG protocol implementations
    circuits/        # Qiskit circuit constructors
    entropy/         # Entropy estimators
    extractors/      # Von Neumann and hash extractors
    noise/           # Quantum noise channel models
    statistics/      # Statistical test suite
    visualization/   # Plotting utilities
    backend/         # Simulator and IBM Quantum backend
  dashboard/        # Streamlit interactive app
  notebooks/        # Jupyter analysis notebooks
  docs/
  tests/
```

3 Module 1 — Multiple QRNG Protocols

3.1 A. Hadamard QRNG

The simplest QRNG protocol prepares a qubit in $|0\rangle$, applies the Hadamard gate, and measures in the computational basis.

Definition: Hadamard Gate

The Hadamard gate H acts on a single qubit as:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad H|0\rangle = |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad H|1\rangle = |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

The circuit for n independent bits is:

$$|0\rangle^{\otimes n} \xrightarrow{H^{\otimes n}} |+\rangle^{\otimes n} \xrightarrow{\text{measure}} x \in \{0, 1\}^n$$

Each bit is independently and uniformly distributed, yielding maximal Shannon entropy $H = n$ bits. The implementation lives in `src/generators/hadamard_qrng.py` and `src/circuits/hadamard.py`.

3.2 B. Bell-State QRNG

The Bell-state protocol uses two-qubit entanglement. A Bell pair (specifically $|\Phi^+\rangle$) is prepared and one qubit is measured.

Definition: Bell States

The four maximally entangled two-qubit Bell states are:

$$|\Phi^\pm\rangle = \frac{|00\rangle \pm |11\rangle}{\sqrt{2}}, \quad |\Psi^\pm\rangle = \frac{|01\rangle \pm |10\rangle}{\sqrt{2}}$$

The preparation circuit applies H to the first qubit followed by a CNOT:

$$|00\rangle \xrightarrow{H \otimes I} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |0\rangle \xrightarrow{\text{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}} = |\Phi^+\rangle$$

Measuring either qubit yields a uniformly random bit. The correlations between the two qubits can serve as a consistency check: if both qubits are measured, a violation of a Bell inequality (such as CHSH) certifies device-independent randomness. The implementation lives in `src/generators/bell_qrng.py` and `src/circuits/bell_state.py`.

3.2.1 CHSH Inequality

The CHSH inequality provides a classical bound on correlations:

$$S = \langle AB \rangle - \langle AB' \rangle + \langle A'B \rangle + \langle A'B' \rangle \leq 2$$

where A, A' and B, B' are dichotomic observables (± 1) on Alice's and Bob's qubits respectively. Quantum mechanics allows $S \leq 2\sqrt{2}$ (Tsirelson's bound), and violation of $S > 2$ certifies genuine entanglement and device-independent randomness.

3.3 C. Angle-Based QRNG

The angle-based protocol prepares the state $R_y(\theta)|0\rangle$ for a rotation angle $\theta \in [0, 2\pi)$, enabling study of how the output probability distribution varies continuously with θ .

Definition: R_y Rotation Gate

$$R_y(\theta) = \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}$$

Acting on $|0\rangle$:

$$R_y(\theta)|0\rangle = \cos \frac{\theta}{2}|0\rangle + \sin \frac{\theta}{2}|1\rangle$$

The measurement probabilities are:

$$P(\text{outcome} = 0) = \cos^2 \frac{\theta}{2}, \quad P(\text{outcome} = 1) = \sin^2 \frac{\theta}{2}$$

This gives a biased QRNG for $\theta \neq \pi/2$, with uniform output only at $\theta = \pi/2$ (recovering the Hadamard protocol). The angle-based generator lives in `src/generators/biased_qrng.py`.

Result: Entropy as a Function of θ

The Shannon entropy of the angle-based generator as a function of θ is:

$$H(\theta) = -\cos^2 \frac{\theta}{2} \log_2 \cos^2 \frac{\theta}{2} - \sin^2 \frac{\theta}{2} \log_2 \sin^2 \frac{\theta}{2}$$

This is maximised at $\theta = \pi/2$ (i.e. $H = 1$ bit) and equals zero at $\theta \in \{0, \pi, 2\pi\}$.

4 Module 2 — Noise Modeling

Real quantum devices are subject to decoherence and gate errors. QREntropy models five standard quantum noise channels, implemented in `src/noise/`.

4.1 Quantum Noise Channels

A quantum channel $\mathcal{E}$ is a completely positive, trace-preserving (CPTP) map on density matrices. It is described by Kraus operators $\{K_i\}$ satisfying $\sum_i K_i^\dagger K_i = I$:

$$\mathcal{E}(\rho) = \sum_i K_i \rho K_i^\dagger$$

4.1.1 Bit-Flip Channel

The bit-flip channel flips $|0\rangle \leftrightarrow |1\rangle$ with probability p :

$$K_0 = \sqrt{1-p} I, \quad K_1 = \sqrt{p} X, \quad \mathcal{E}_{\text{bf}}(\rho) = (1-p)\rho + p X \rho X$$

4.1.2 Phase-Flip Channel

The phase-flip channel applies a Z gate with probability p , leaving the computational basis untouched but damaging superposition:

$$K_0 = \sqrt{1-p} I, \quad K_1 = \sqrt{p} Z, \quad \mathcal{E}_{\text{pf}}(\rho) = (1-p)\rho + p Z \rho Z$$

4.1.3 Depolarizing Channel

The depolarizing channel is the most common single-qubit noise model. With probability p the qubit is replaced by the maximally mixed state:

$$\mathcal{E}_{\text{dep}}(\rho) = (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z)$$

For large p , the output approaches $\frac{I}{2}$, which has maximal entropy but is completely uncertifiable.

4.1.4 Amplitude Damping

Amplitude damping models energy relaxation (T_1 decay), the process by which $|1\rangle \rightarrow |0\rangle$ with probability γ :

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \quad K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix}$$

This channel biases measurements toward $|0\rangle$, reducing entropy.

4.1.5 Measurement Error

Measurement errors are modelled by a readout confusion matrix M :

$$\begin{pmatrix} P(\text{read } 0) \\ P(\text{read } 1) \end{pmatrix} = \begin{pmatrix} 1 - \epsilon_0 & \epsilon_1 \\ \epsilon_0 & 1 - \epsilon_1 \end{pmatrix} \begin{pmatrix} P(\text{true } 0) \\ P(\text{true } 1) \end{pmatrix}$$

where ϵ_0 and ϵ_1 are the probabilities of misidentifying $|0\rangle$ and $|1\rangle$ respectively.

4.2 Noise vs. Entropy: Research Question

The central research question of QREntropy concerns how each noise channel degrades entropy at varying noise strengths:

- For amplitude damping and bit-flip, entropy *may decrease* as output is biased toward $|0\rangle$ or $|1\rangle$
- For depolarizing noise at high p , entropy *may appear maximal* but the randomness is classical (thermal), not certified
- Randomness extraction can recover certified randomness if the min-entropy of the raw output exceeds the extraction threshold

5 Module 3 — Entropy Estimation

Entropy quantifies the amount of uncertainty (and hence potential randomness) in a distribution. QREntropy computes three complementary entropy measures, implemented in `src/entropy/`.

5.1 Shannon Entropy

Definition: Shannon Entropy

For a discrete random variable X with probability distribution $\{p_i\}$:

$$H(X) = - \sum_i p_i \log_2 p_i \quad (\text{bits})$$

where $0 \log_2 0 := 0$. Shannon entropy ranges from 0 (deterministic) to $\log_2 |\mathcal{X}|$ (uniform).

For a fair coin (QRNG with $\theta = \pi/2$): $H = -(0.5) \log_2(0.5) - (0.5) \log_2(0.5) = 1$ bit.

5.2 Min-Entropy

Min-entropy is the most conservative entropy measure, dominated by the highest-probability outcome. It lower-bounds the amount of randomness that can be certified and extracted.

Definition: Min-Entropy

$$H_\infty(X) = -\log_2 \left(\max_i p_i \right)$$

Min-entropy is relevant for cryptographic applications: H_∞ bits of min-entropy guarantee that no single outcome occurs with probability greater than 2^{-H_∞} .

5.3 Collision Entropy (Rényi Entropy of Order 2)

Definition: Collision Entropy

$$H_2(X) = -\log_2 \left(\sum_i p_i^2 \right)$$

The quantity $\sum_i p_i^2$ is the collision probability — the probability that two independently drawn samples from X are equal. H_2 is related to the security of universal hashing and lies between Shannon and min-entropy: $H_\infty \leq H_2 \leq H$.

5.4 Rényi Entropy Family

All three measures are special cases of the Rényi entropy of order α :

$$H_\alpha(X) = \frac{1}{1-\alpha} \log_2 \left(\sum_i p_i^\alpha \right)$$

with $H_1 = H$ (Shannon), H_2 (collision), and $H_\infty = \lim_{\alpha \rightarrow \infty} H_\alpha$ (min-entropy).

6 Module 4 — Randomness Extraction

Raw quantum bit strings from a noisy device may be biased or partially correlated. Randomness extractors are deterministic (or seeded) functions that map a weakly random source with high enough min-entropy to a nearly uniform output. Implementations are in `src/extractors/`.

6.1 Von Neumann Extractor

The Von Neumann extractor processes consecutive non-overlapping pairs of bits:

$$(0, 1) \rightarrow 0, \quad (1, 0) \rightarrow 1, \quad (0, 0) \rightarrow \text{discard}, \quad (1, 1) \rightarrow \text{discard}$$

Result: Von Neumann Correctness

If the source produces i.i.d. bits with $P(0) = p$ and $P(1) = 1 - p$ (with $p \neq 0, 1$), then retained pairs (01) and (10) each occur with probability $p(1 - p)$, and the extractor output is exactly uniform regardless of p .

Trade-off: The extraction rate (output bits per input bit) equals $2p(1 - p)$, which is maximised at $p = 0.5$ (yielding rate 0.5) and tends to zero as $p \rightarrow 0$ or $p \rightarrow 1$. The extractor discards roughly half of all bits even under ideal conditions.

6.2 SHA-256 Hash Extractor

A cryptographic hash function $h : \{0, 1\}^n \rightarrow \{0, 1\}^{256}$ applied to a long raw bit string serves as a seeded strong extractor. By the leftover hash lemma:

Result: Leftover Hash Lemma (informal)

If the raw source has min-entropy $H_\infty \geq k$ over n bits, and a universal hash family is used with output length $m \leq k - 2 \log(1/\varepsilon)$, then the output is ε -close to uniform in total variation distance.

SHA-256 is not a universal hash family in the strict sense, but is computationally indistinguishable from one for practical purposes, making it a strong practical extractor.

6.3 Comparison: Raw vs. Extracted Entropy

The framework computes and plots:

$$H_\infty(\text{raw}) \xrightarrow{\text{Von Neumann or SHA-256}} H_\infty(\text{extracted}) \approx 1$$

The gap between these quantities measures both the severity of noise and the effectiveness of the extractor.

7 Module 5 — Statistical Test Suite

The statistical test suite evaluates whether the generated bit string is consistent with a true uniform i.i.d. source. All tests are implemented in `src/statistics/` with automated report generation.

7.1 Frequency / Monobit Test

The frequency test checks whether the proportion of 1s is close to $1/2$:

$$\hat{p} = \frac{\#\{1s\}}{n}, \quad Z = \frac{|\hat{p} - 0.5|}{\sqrt{1/(4n)}} \sim \mathcal{N}(0, 1) \text{ under } H_0$$

A p -value below significance level $\alpha = 0.01$ indicates bias.

7.2 Chi-Square Goodness-of-Fit Test

For blocks of k bits, the chi-square test compares observed block frequencies O_i to the expected frequencies $E_i = n/2^k$:

$$\chi^2 = \sum_{i=1}^{2^k} \frac{(O_i - E_i)^2}{E_i} \sim \chi_{2^k-1}^2$$

7.3 Runs Test

A *run* is a maximal sequence of identical bits. The runs test checks whether the number of runs R in a sequence of length n is consistent with independence:

$$\mathbb{E}[R] = \frac{2n\hat{p}(1-\hat{p}) \cdot n}{n} + 1 - 2\hat{p}^2 - 2(1-\hat{p})^2$$

Too few runs indicate positive autocorrelation; too many indicate negative autocorrelation.

7.4 Autocorrelation Test

The lag- ℓ autocorrelation of the bit sequence $(x_1, \dots, x_n)$ (with $x_i \in \{-1, +1\}$) is:

$$C(\ell) = \frac{1}{n-\ell} \sum_{i=1}^{n-\ell} x_i x_{i+\ell}$$

For a uniform i.i.d. source, $\mathbb{E}[C(\ell)] = 0$ for all $\ell \geq 1$. The framework plots $C(\ell)$ vs. ℓ with 95% confidence bands.

7.5 Serial Test (Digram Test)

The serial test examines all 2^k possible length- k patterns in overlapping substrings, testing whether each pattern appears with equal frequency $1/2^k$. It is a generalisation of the chi-square test to overlapping blocks.

7.6 Poker Test

The bit string is divided into non-overlapping k -bit blocks. Let f_j be the count of the j -th k -bit pattern. The test statistic is:

$$T = \frac{2^k}{n} \sum_{j=0}^{2^k-1} f_j^2 - n$$

7.7 NIST SP 800-22 Alignment

These tests are a subset of the NIST SP 800-22 statistical test suite, the standard reference for RNG evaluation in cryptographic applications. QREntropy generates an automated summary report giving pass/fail decisions for each test, p -values, and commentary.

8 Module 6 — IBM Quantum Integration

QREntropy supports two execution backends, implemented in `src/backend/`.

8.1 Simulator Backend

The `AerSimulator` from Qiskit Aer provides noiseless and noise-model simulation:

- **Statevector simulation:** exact probability amplitudes; ideal randomness
- **Shot-based simulation:** finite-sample approximation to the true distribution
- **Noise model injection:** Qiskit `NoiseModel` allows adding the channels from Module 2 to simulation

8.2 IBM Quantum Hardware Backend

Via the Qiskit IBM Runtime (`qiskit-ibm-runtime`), circuits can be transpiled and submitted to real quantum processors. Hardware results include:

- Native gate decomposition and qubit mapping
- Realistic T_1 , T_2 decoherence, gate errors, and readout errors
- CLOPS (Circuit Layer Operations Per Second) performance metrics

8.3 Simulator vs. Hardware Comparison

A key scientific output of QREntropy is a systematic comparison:

$$\text{Ideal Simulator} \longleftrightarrow \text{Noisy Simulator} \longleftrightarrow \text{IBM Hardware}$$

For each QRNG protocol and noise level, the framework reports entropy measures and statistical test p -values side by side, quantifying how well noisy simulation predicts real hardware behaviour.

9 Module 7 — Interactive Dashboard

The Streamlit dashboard (in `dashboard/`) provides a parameter-driven interface for non-expert users:

1. **Choose protocol** — Hadamard, Bell-state, or angle-based (θ slider)
2. **Number of qubits** — 1 to 10
3. **Shots** — 100 to 100 000
4. **Noise channel and level** — drop-down selector + strength slider
5. **Backend** — Simulator or IBM Quantum
6. **Generate** — submit circuit and collect results
7. **View statistics** — real-time entropy plots, test suite results, histograms
8. **Download report** — PDF summary with all metrics

The dashboard makes QREntropy accessible as a teaching and demonstration tool without requiring users to interact with the Python API directly.

10 Module 8 — Visualization

The `src/visualization/plots.py` module generates a standardised set of figures for all protocols and noise levels:

- **Circuit diagrams** — rendered via Qiskit’s `circuit.draw()` or Matplotlib backends
- **Output histograms** — bar charts of bitstring frequencies, overlaid with expected uniform distribution
- **Entropy vs. noise** — line plots of H , H_2 , and H_∞ as functions of noise parameter p or γ
- **Zero/one frequency plots** — per-qubit bias visualisation
- **Autocorrelation plots** — $C(\ell)$ vs. lag ℓ with confidence bands
- **Heatmaps** — joint distributions for multi-qubit outputs
- **Bell-state correlation diagrams** — CHSH correlation matrices for Bell-state protocols under varying noise

11 Module 9 — Benchmarking

To contextualise QRNG performance, QREntropy benchmarks quantum generators against classical pseudorandom sources.

Metric	Python random	NumPy	Quantum (sim)	Quantum (hw)
Shannon entropy	≈ 1	≈ 1	≈ 1	< 1 (noisy)
Min-entropy	≈ 1	≈ 1	≈ 1	< 1
Bias ($ \hat{p} - 0.5 $)	~ 0	~ 0	~ 0	> 0
Chi-square p -value	pass	pass	pass	varies
Runs test p -value	pass	pass	pass	varies
Speed (Mbits/s)	very high	very high	moderate	low
Certification	none	none	sim-limited	device-dep.

Table 1: Benchmarking summary: QREntropy compares classical PRNGs with quantum generators across entropy, bias, and statistical test metrics.

Classical PRNGs are fast and statistically uniform, but are deterministic and non-certifiable. Quantum generators on real hardware exhibit lower throughput but offer physical certification guarantees — degraded by noise, partially recoverable by extraction.

12 Module 10 — Research Component

12.1 Primary Research Question

How do quantum noise channels affect the statistical quality and entropy of quantum-generated random numbers, and can randomness extraction recover high-quality randomness from noisy quantum measurements?

12.2 Experimental Design

For each combination of $\{\text{protocol}\} \times \{\text{noise channel}\} \times \{\text{noise level } p \text{ or } \gamma \in [0, 0.5]\}$:

1. Generate $N = 10\,000$ bits
2. Compute H, H_2, H_∞ on the raw output
3. Apply Von Neumann and SHA-256 extraction
4. Compute H, H_2, H_∞ on the extracted output
5. Run the full statistical test suite on both raw and extracted bits
6. Repeat on IBM hardware at matching noise levels where feasible

12.3 Expected Outcomes

- **Amplitude damping** is expected to produce the steepest entropy degradation, since it systematically biases toward $|0\rangle$
- **Depolarizing noise** may paradoxically maintain high Shannon entropy while destroying certifiability
- **Bell-state QRNG** is expected to degrade faster under two-qubit gate noise than single-qubit Hadamard QRNG
- **SHA-256 extraction** is expected to recover uniform statistics at higher noise levels than Von Neumann extraction, at the cost of requiring a longer raw input
- **Real hardware** results are expected to show systematic deviations from depolarizing-only noise models, motivating more sophisticated device characterisation

13 Summary and Future Directions

QREntropy provides a comprehensive research framework spanning:

- Three QRNG protocols with controllable bias via $R_y(\theta)$
- Five quantum noise channels with tunable strength
- Three information-theoretic entropy estimators
- Two randomness extraction strategies
- A seven-test statistical evaluation suite
- Simulation and real IBM Quantum hardware backends
- An interactive Streamlit dashboard and automated reporting

Potential extensions include:

- Device-independent QRNG via full CHSH Bell test with loophole-free settings
- Semi-device-independent protocols using entropic uncertainty relations
- Integration with quantum error correction (stabiliser codes) to protect the QRNG circuit

- Extending to qutrits or higher-dimensional systems for higher bit-per-measurement rates
- Continuous-variable (CV) QRNG using homodyne measurement of vacuum fluctuations

Note

This document reflects the project architecture and theoretical foundations as of June 2026. The project is under active development; module implementations and experimental results will be reported in dedicated notebooks and a final write-up.